COMPANY NAME/LOGO

SDK/API – Viewer URL Support

Developer's Guide

Supported Versions: 6.1, 6.2

Last Revised: April 7, 2026

© YEAR, PRODUCT NAME Corporation dba PRODUCT NAME Corporation. All rights reserved.

DISCLAIMER

PRODUCT NAME has made every effort to ensure that this document is accurate.
PRODUCT NAME disclaims liability for any inaccuracies or omissions.

No part of the contents of this publication may be reproduced, transmitted, transcribed, stored in a retrieval system, or translated into any language in any form without written consent from PRODUCT NAME Corporation.

Information in this document is subject to change without notice and
does not represent commitment on the part of PRODUCT NAME.

If you find information in this document that is incorrect, misleading, or incomplete,
we would appreciate your comments and suggestions.

Table of Contents

Purpose of this Guide 1

PRODUCT NAME SDK Overview 1

SDK/API System Overview 2

Application Command Line Arguments 3

Basic Command Line Argument Used for Both Tool Name and Tool Name 3

Working with Tool Name 5

Working with Tool Name 6

PRODUCT NAME SpeedLoader 7

Working with SpeedLoader 7

Setting Up and Using the PRODUCT NAME URL API 9

System Access 9

Searches 10

Viewer Integration 11

PRODUCT NAME SOAP Services API 13

PRODUCT NAME Application Server 13

PRODUCT NAME DocumentServer 13

Working with PRODUCT NAME SOAP Services 14

PRODUCT NAME ViewerJavaScript API 16

Access the JavaScript Client Integration Files 16

Requirements for Viewer Integration 17

Steps to Integrate the HTML5 Viewer 17

PRODUCT NAME REST Services API 18

Working with PRODUCT NAME REST API Services 18

Appendices 20

Appendix A – Backward Compatibility & IIS Configuration 21

Appendix B – Examples of Invalid Command Line Parameters 22

Revision History

Item #	Release #	Revision Date	Description	Tracking Notes
005	ALL	03/28	Revised to apply redactions as needed for portfolio (all pages).	• KB
002	6.1 & 6.2	02/09/18	• Added Viewer Integration section (beginning on page 11 of this document) • Added Security Services for REST services.	• Item 26724 • Requested by Lopez during scrum on 02/08
001		12/8/17	Added information for URI format exception for Viewer to page 11 of this document	Item 26069
000		11/17	ORIGINAL	US 24845

© 2018, PRODUCT NAME Corporation dba PRODUCT NAME Corporation.

Purpose of this Guide

The intended audience for this guide is software development professionals. Beginning with less technical information (such as interactions with PRODUCT NAME via command line usage of products), this guide becomes progressively more technical. Software development professionals who review this document will then have the necessary information to allow for the implementation of customized, integrated solutions.

An **SDK** is a set of software development tools that allows for the creation of applications in association with a specific software package. This means that software developers can create and implement customized functionality with PRODUCT NAME: meeting specific business needs.

An **API** is an application programming interface that allows software developers to write code that consumes the API. By consuming the API, developers can integrate software systems together without having to know or understand the inner workings of the software system being consumed.

Overview

The PRODUCT NAME SDK provides a variety of technology options allowing interaction or integration between PRODUCT NAME and other software systems.

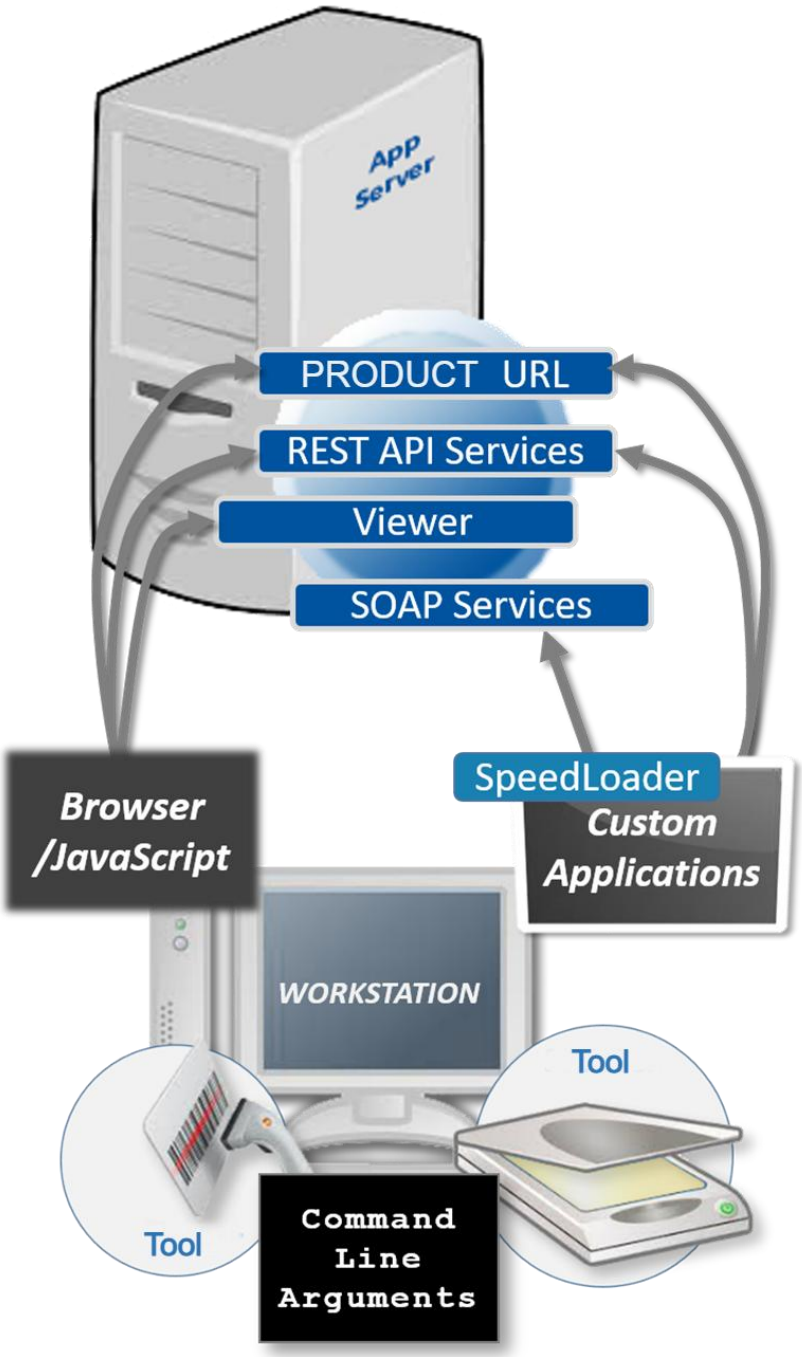
Using the SDK, you can easily build applications that provide the following capabilities:

- Document management, search, retrieval, and viewing
- Work item search
- Scanning
- Barcode processing
- Document and metadata import/export
- Business process management

The PRODUCT NAME SDK supports the following technologies:

- API Services – Representational State Transfer API that provides access to PRODUCT NAME logic and data
- TBD

SDK/API System Overview



Application Command Line Arguments

Basic Command Line Argument Used for Both Tool Name and Tool Name

There are seven basic command argument elements used for Tool Name and Tool Name:

Argument Elements	Definition
-a	Load PRODUCT NAME Application
-d	PRODUCT NAME Web Server System Alias ▪ This name appears within the Server name that appears in the drop-down list on the PRODUCT NAME login screen. ▪ This element crosses references and uses the associated web server in the application.
-f	Load file in PRODUCT NAME
-i	PRODUCT NAME Index Names and Values
-p	PRODUCT NAME User Password
-u	PRODUCT NAME User
-w	Attach PRODUCT NAME Web Server

- If the User Name (-u), Password (-p), WebServer (-w) or Application ID (-a) parameters are missing on the command line, a message displays an error, and command line parameters are not processed.
- If any of the command line parameters are invalid, a message appears describing the error; and command line parameters are not processed.

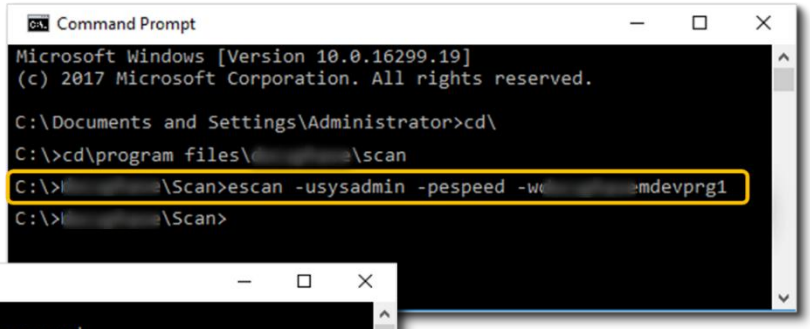
Working with Login Credential Line Arguments

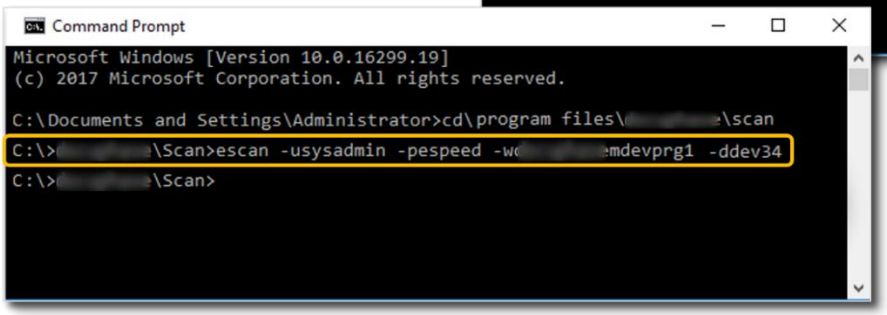
Proper credentials are required for any operation in any application. In fact, logging into either Tool Name or Tool Name is automatic when the associated user and server credentials are entered. However, while automatic login is not unusual, there are other forms of command line options that can be used:

If Single Sign-On Mode is…	If Alias Server is…	Use this Command Line Argument…
OFF	DEFINED	**–u, -p and –w**
OFF	UNDEFINED	**–u, -p, –w, and -d**
ON	DEFINED	**–u, -w, and -–d**
ON	UNDEFINED	**–u and –w**

EXAMPLES

Here, the command line argument displays automatic submission of credentials when not in single sign-on mode, using the web server name:





While here, the command line argument displays an automatic submission of credentials when in single sign-on mode using an alias.

Command Line Argument for both Tool Name and Tool Name (continued on next page)

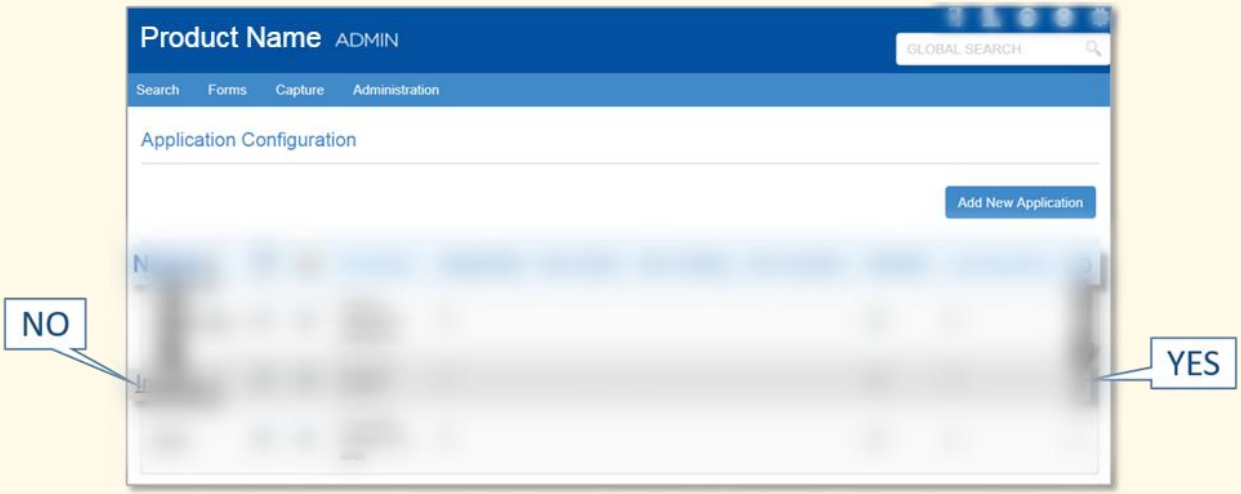
Command Line Argument for Both Tool Name and Tool Name (continued)

Creating/Editing Application, Indexes, and Index Value Line Arguments

Besides using the basic command line argument(shown on the previous page), settings can be created that automatically load the PRODUCT NAME application, as well as its index values:

IMPORTANT!

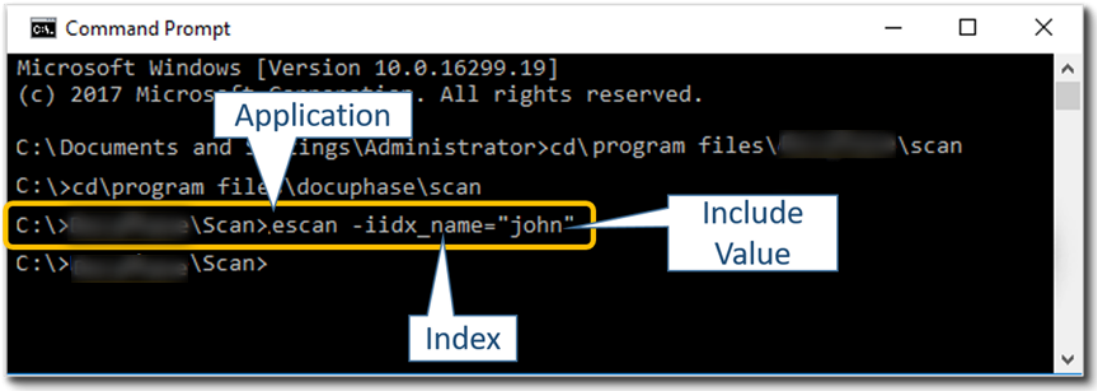
- ***The command line index arguments use the Application ID, rather than the PRODUCT NAME names. In other words, if you want to load an application named in PRODUCT NAME as "Invoice", do not use the name "Invoice"; instead, use the application ID found at Admin▶Application.***



- ***You need to add the "–i" command for each index you want to add and define from the associated applications. Multiple index values can be included in a command line argument.***

EXAMPLE

This example provides you with an idea of how the indexes must be entered.



Working with Tool Name

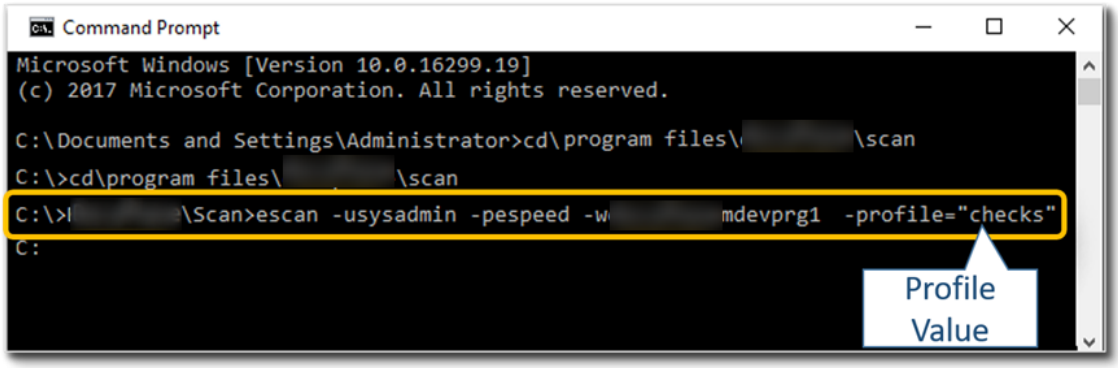
Setting Up Tool Name-specific Line Arguments (File Names and Profiles)

In addition to the Basic command line argument(as shown on page 3 of this guide), you can use others argument elements to automatically load file names and profiles for Tool Name.

Argument Elements	Definition
-f	Load file name
-profile	Load Tool Name profile (please refer to Tool Name user documentation for more information about profiles)

EXAMPLE

This example provides you with an idea of how the indexes must be entered.



Working with Tool Name

Process Behaviors

- Only one Tool Name process runs at a time. Subsequent Tool Name processes pass the command line parameter values to the existing Tool Name process and then terminate.
- Any data displayed on the existing Tool Name screen is replaced by command line parameter values that are passed. If a subsequent Tool Name process has no command line parameters, it terminates, having no effect on the existing Tool Name process currently running.
- The Command Line Processing check box appears at the top of the screen when an existing Tool Name process receives command line parameters.
- When command line processing is in effect, the application selection drop-down list is disabled (see *Error! Reference source not found.* shown below).
- When Tool Name receives field values from the command line parameter (-i), it displays any values passed, and the cursor positions itself on the first field on the screen.
- Within the **Tools>Display Fields** settings:
  - Fields set to display by default display regardless of whether or not the field is passed from the command line parameters.
  - Fields set to ***not*** display by default are only displayed if passed from the command line parameters.

Setting New Default Values

In general, settings are not saved as new default values for subsequent barcode pages. To set new default values, follow the procedure below:

1) Deselect the Command Line processing option to disable command line processing.
2) Make changes to Print/Page settings or Tools settings.
   - *The Tool Name processes normally and saves changes made as new defaults for subsequent barcode pages.*

Selecting Another Application

1) Deselect the Command Line processing option to disable command line processing, remove the checkbox, then enable the drop-down list.

To use a database lookup for a passed lookup field value:

2) Position the cursor over the lookup field, then press the TAB key to initiate a database lookup and override values passed from the command line.

-OR-

To make other field value changes (without performing a database lookup):

Click in the field you want to change (DO NOT PRESS TAB), then make the change(s).

- *Repeat this step for all fields you want to edit.*

-OR-

To print/preview the barcode page (without performing a database lookup):

Click the **Print** or **Preview** options (DO NOT PRESS TAB).

IMPORTANT!
When command line processing is in effect, any changes to Print/Page settings or changes made under the Tools menu item are only in effect for the current barcode page to be printed.

TIPS
- ***To keep the data passed, select a field or the Print/Preview option.***
- ***Press the TAB key while in a lookup field to override any data passed with database values.***

PRODUCT NAME SpeedLoader

Working with SpeedLoader

SpeedLoader is a .NET assembly (i.e., partially compiled code used for deployment) to import large numbers of documents into PRODUCT NAME, in a transactional manner (i.e., at least partially in a sequential order to maintain consistency). In addition, SpeedLoader uses a specific import file as the source for documents to be imported, allowing for rollbacks as needed (see *NOTE* below).

> ***NOTE***
> *Using an import file as a source allows for rollbacks of documents included in a failed import process. Imports using SpeedLoader are all or nothing; if any document in an import fails, the entire import is rolled back.*

Setting Up the Import File

The following are all characteristics of the SpeedLoader Import file (as described above):

- An XML file that defines the import and each document
- The first parameter is an import filename
- It contains two sections:
  - **Configuration** - appears once per file and contains configuration information. It also contains the following required elements:

Elements	Definition
Server	• The name of the PRODUCT NAME authentication server. • The value can be an FQDN (Fully Qualified Server Name) or an ordinary server name. • If the value is NOT an FQDN, then the assumed authentication is as follows: http://{server name/applicationserver/authenticationservice.asmx
Table_name	The name of the table into which a document is being placed
Status	The status assigned to the newly imported documents (e.g., XX, XX, XX)
Username	The name of the user to whom the newly imported documents are assigned.

  - **Document** - appears once per file and can contain multiple document elements that describe the document being imported. Each document element is made up of one or more field elements that contain the following required attributes:

Elements	Definition
Name	The name of the column in the PRODUCT NAME table into which the document is being placed
Type	The type of column (i.e., float, int, string, and date) into which a document is being placed
Value	• The value being placed in the above-named column • All values must be valid for conversion to the specified type (as described above) • Dates must be in the format "YYYMMDD"
itemcount	Informs the system of the number of document elements designated
doc_file_name	This points to the file belonging to the document element

EXAMPLE

```
<data version="1.0.0" type="      .SpeedLoader">
  <configuration>
      <server>localhost</server>
      <table_name>_obj_2</table_name>
      <status>E</status>
      <username>sysadmin</username>
  </configuration>
  <documents>
      <document>
         <fields>
            <field name="idx_name" type="string" value="john" />
            <field name="idx_amount" type="float" value="123.45" />
            <field name="idx_count" type="int" value="100" />
            <field name="idx_date" type="date" value="20040609" />
         </fields>
         <doc_file_name>c:\dir\filename00001.tif</doc_file_name>
      </document>
  </documents>
  <itemcount>1</itemcount>
</data>
```

Working with SpeedLoader (continued on next page)

Working with SpeedLoader (continued)

Making Pre-Import Confirmations

Before SpeedLoader attempts to import documents, it automatically performs precise processes to help prevent any import from failing. These processes include the following:

- Each import file is checked to verify that all index values can be converted to the designated type. If any value fails the conversion, SpeedLoader doesn't attempt the import; validation includes checking string field length.
- The total file size for all documents is calculated, then doubled, and submitted for import to make sure that there is enough space to complete the import process. If SpeedLoader determines that there isn't enough space for the import of all the documents, the import is not performed.

Using SpeedLoader Interface Command Elements

The Interface Command is a combination of the following elements:

Elements	Definition
FileName	Always the first parameter of the command, and is a required element
Username	The PRODUCT NAME user to impersonate
Password	The password associated with the user (as described above)
Instance	An optional value that corresponds to the instance names in the PRODUCT NAME authentication server. If no Instance value is provided, the "Default" instance is used.

EXAMPLE
Here's how the interface command elements appear:

```
SpeedLoader.Process(string filename, string username, string password, string instance)
```

Setting Up and Using the PRODUCT NAME URL API

This section describes how the system provides third-party user (i.e., someone outside your designated user group) access to one or more documents from within the PRODUCT NAME platform. In this process, the PRODUCT NAME URL automatically logs the user into the system, allowing them to search for (and select) a document that is then retrieved from the PRODUCT NAME server. The document is then rendered and displayed in the PRODUCT NAME Viewer (also see *NOTES* below).

- System access to users (i.e., logging in) – see below
- Searches – see page 10
- Viewer Integration – see page 11

NOTES

- *PRODUCT NAME can interface with any ODBC-compliant database for data exchange and third-party product integration.*
- *The Document Viewer can be embedded into a third-party application, giving users PRODUCT NAME Viewer Toolbar functionality.*
- *See Appendix A on page 21 of this document for information about Backward Compatibility.*

System Access

The login prompt appears when a user is not already logged into PRODUCT NAME. In order for a third-party user to bypass this page, and continue through to the requested search, the user must include the key/values associated with the Access.aspx page within the QueryString (see details below).

Access.aspx

Querystrings must include the following:

- **Action** = Login (required)
- **UserName** = USERNAME (required)
- **Password** = PASSWORD (required) OR EncryptedPassword = ENCRYPTEDPASSWORD

EXAMPLE

- http://SERVERNAME/PRODUCT NAME38/web/authorization/Access.aspx?Action=login&UserName=USERNAME&Password=PASSWORD
- http://SERVERNAME/PRODUCT NAME38/web/authorization/Access.aspx?Action=login&UserName=USERNAME&EncryptedPassword=ENCRYPTEDPASSWORD

HandlerPage.aspx

The HandlerPage is the component that processes all queries from outside of PRODUCT NAME. This is the page that is accessed using appropriate parameters, including parameters for logging in, as well as for searching (see **Searches** section on page 10 for more about each search, and its required parameters).

EXAMPLE

- http://SERVERNAME/PRODUCT NAME38/web/handler/HandlerPage.aspx?Action=login&UserName=USERNAME&PASSWORD=PASSWORD&ApplicationID=2
- http://SERVERNAME/PRODUCT NAME38/web/handler/HandlerPage.aspx?Action=login&UserName =USERNAME&Password=PASSWORD&ApplicationID=2&IDX_Name=fredex

Searches

Third-party users can utilize a set of QueryString values that perform searches, just as if the user were searching from inside PRODUCT NAME. There are three types of searches a user can perform:

- **"CrossQuery" Search** – performs a search across all applications based on a specific index value entered in the QueryString
- **Global Search** – performs a search of every document, in every application, for specific keyword values entered in the QueryString
- **Generic Search** – performs a search using a combination of Application, ObjectID, index value, and/or index value filter according to the values entered in the QueryString

IMPORTANT!
Please remember… Entries for Search and Indexes ARE CASE SENSITIVE!

Search QueryString Values

Search Type	Querystring Value*	Required Values
CrossQuery	**index**, **search**	Values shown here in **red** are required.
Global	**gsearch**	
Generic	**ApplicationID**, IDX_IndexName, ObjectID	

Search Hierarchy

Searches are processed in the following order:

- **1st** - CrossQuery search
- **2nd** - Global search
- **3rd** - Generic search

EXAMPLE
IF the following query was entered:
http://SERVERNAME/PRODUCT 38/web/handler/HandlerPage.aspx?Action=login&USERNAME=USERNAME&PASSWORD=PASSWORD&ApplicationID=2&gsearch=Apple&search=Orange&index=IDX_Fruit
THEN the cross query result will be recognized and executed, whereas the global search will not be recognized or executed.

In/Not in Searches

Within the search querystrings, you can specify the inclusion or exclusion of certain items included in the search results:

To…	Use this Querystring Value
Include results	IDX_INDEXNAME=in(value1,value2,...,valueN)
Exclude results	IDX_INDEXNAME=not in(value1,value2,...,valueN)

EXAMPLE

- *To include documents from the White Orchid and Fredex applications, with a status of "paid" enter the following:* http://SERVERNAME/PRODUCT NAME38/web/handler/HandlerPage.aspx?Action=login&UserName=USERNAME&Password=PASSWORD&ApplicationID=17&IDX_name=in(white orchid,fredex)&idx_status=in(paid)
- *To include documents from any applications except those associated with "Dayback" and "Fredex" enter the following:* http://SERVERNAME/PRODUCT NAME38/web/handler/HandlerPage.aspx?Action=login&UserName=USERNAME&Password=PASSWORD&ApplicationID=2&IDX_Name=not in(dayback,fredex)
- *To include documents from the Fredex application only, enter the following:* http://SERVERNAME/PRODUCT NAME38/web/handler/HandlerPage.aspx?Action=login&UserName=USERNAME&Password=PASSWORD&ApplicationID=2&IDX_Name=in(fredex)

Searches (continued on next page)

Searches (continued)

Required Licensing

To maintain the integrity of the PRODUCT NAME system, as well as to safeguard client information, the SDK URL-based searching requires a license. The product under which this URL search is licensed is "Partner Extension", and includes the following properties:

Search Type	Status	Property
CrossQuery	Pending Approval	CrossQuerySearchEnabled
Global	Pending Approval	GlobalSearchEnabled
Generic	Current standard search	GenericSearchEnabled

NOTES

- *Names of Search Properties may need to change to obscure identity, if required.*
- *Each property maintains a Boolean value, where:*
  - *'True' means it is enabled.*
  - *'False' means it is disabled.*

Viewer Integration

URL Formats

You can use query string values to load the ActiveX Document Viewer using the following URL formats:

For an application and document with Revision Control…	Use this URL format…
Enabled (☑)	https://server.mydomain.com/PRODUCT NAME/web/ `/authorization/Access.aspx?ReturnUrl=/PRODUCT NAME/web/search/viewer.aspx?appId%3d2%26objectId%3d47%26version%3d0`
Disabled (☐)	https://server.mydomain.com/PRODUCT NAME/web/ `/authorization/Access.aspx?ReturnUrl=/PRODUCT NAME/web/search/viewer.aspx?appId%3d2%26objectId%3d47`

Viewer Integration (continued on next page)

Viewer Integration (continued

Query String Parameters

The query string parameters are as follows:

This value…	Refers to the …
ReturnUrl	Encoded PRODUCT NAME URL that will be loaded (see *IMPORTANT!* note below)
appId	application ID that contains the document.
objectId	object ID (document ID) for the document to be viewed.
version	version number of the document if the application is revision controlled.

IMPORTANT!

The Return URL parameter (as shown above) is encoded to prevent potentially malicious data from being passed to PRODUCT NAME.

The following example demonstrates how the PRODUCT NAME ActiveX Viewer can be loaded from an external software system: producing a seamless integration with PRODUCT NAME.

```
<!DOCTYPE html>

<html lang="en" xmlns="http://www.w3.org/1999/xhtml">
<head>
    <meta charset="utf-8" />
    <title></title>
</head>
<body>

    <script>
        var newWindow;

        //  works
        //newWindow = window.open('https://myServer/PRODUCT
NAME/web/authorization/Access.aspx?ReturnUrl=/PRODUCT NAME/web/search/viewer.aspx?appId%3d2%26objectId%3d47',
'_blank');

        // works
        var newURL = 'https://myServer/PRODUCT NAME/web/authorization/Access.aspx?ReturnUrl=/PRODUCT
NAME/web/search/viewer.aspx?' + encodeURIComponent('appId=2&objectId=47');

        alert(newURL);

        newWindow = window.open(newURL, '_blank');

        newWindow.focus();


    </script>

</body>
</html>
```

PRODUCT NAME SOAP Services API

PRODUCT NAME includes a number of "Servers" that provide one or more SOAP (i.e., Simple Object Access Protocol) services. By utilizing the PRODUCT NAME SOAP services API integrated systems execute logic built in to PRODUCT NAME: ensuring data integrity.

> **NOTE**
> *PRODUCT NAME SOAP Services API is considered a legacy technology, and is, therefore, gradually being deprecated.*

All SOAP services are discoverable and publish WSDL documents. To access the WSDL document for a particular service, use the following URL pattern to get to the WSDL of the corresponding service:

http://localhost/{servername}/{servicename}.asmx?WSDL

PRODUCT NAME Application Server

This section provides a description of each "Server" and its corresponding services. PRODUCT NAME Application Server contains two SOAP services that provide access to application data:

DataService

The DataService SOAP service provides access to retrieve data, as well as some additional service methods that allow for update of system-level information.

To access the DataService WSDL* that describes each service method along with required parameters, use the following URL format:

http://{machine name}/ApplicationServer/DataService.asmx?WSDL

AuthenticationService

The AuthenticationService SOAP service provides methods for authentication and authorization.

To access the AuthenticationService WSDL* that describes each service method along with required parameters, use the following URL format:

http://{machine name}/ApplicationServer/AuthenticationService.asmx?WSDL

PRODUCT NAME DocumentServer

The PRODUCT NAME Document Server contains a single SOAP service called DocumentService that provides access to documents.

Using DocumentService

The DocumentService SOAP service provides secure access to documents in the PRODUCT NAME repository. This service leverages SOAP service standards such as ws-security, ws-policy, and ws-attachments.

To access the DocumentService WSDL* that describes each service method along with required parameters, use the following URL format:

http://{machine name}/DocumentServer/DocumentService.asmx?WSDL

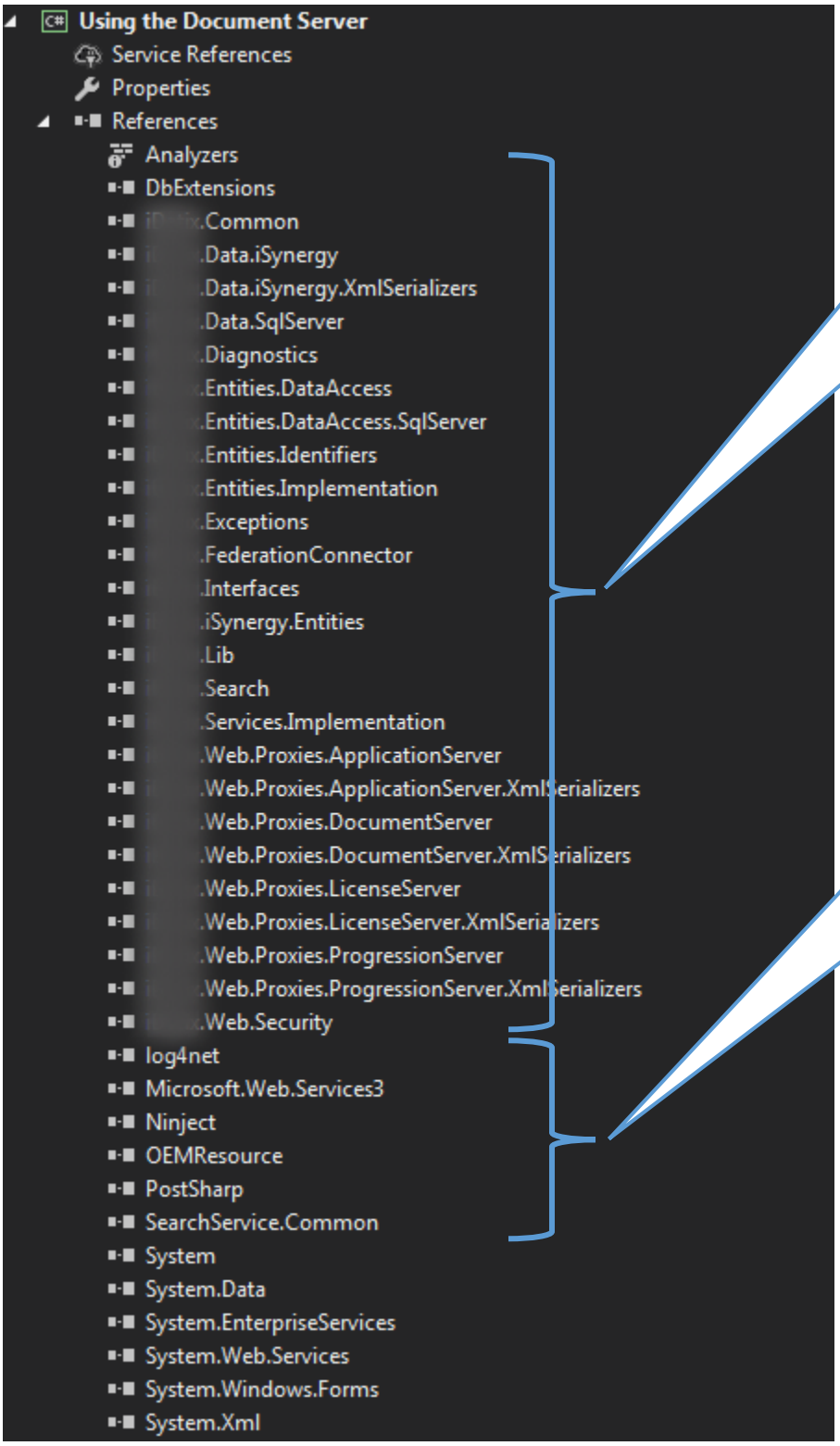
**Please refer to the service WSDL to view details of each method, the input parameters, and a description of what action the method performs.*

Working with PRODUCT NAME SOAP Services

The PRODUCT NAME SDK provides sample code that demonstrates the use of the DocumentServer DocumentService. This sample code is made available as part of an installable MSI package and is provided upon request.

Project References

The project references PRODUCT NAME assemblies required in order for the sample code to compile and execute. These references are shown below:



Required PRODUCT NAME assemblies begin with PRODUCT NAME. naming convention.

DbExtensions is a PRODUCT NAME assembly.

Additional required assemblies log4net, Ninject, PostSharp.

SearchService.Common is a PRODUCT NAME assembly.

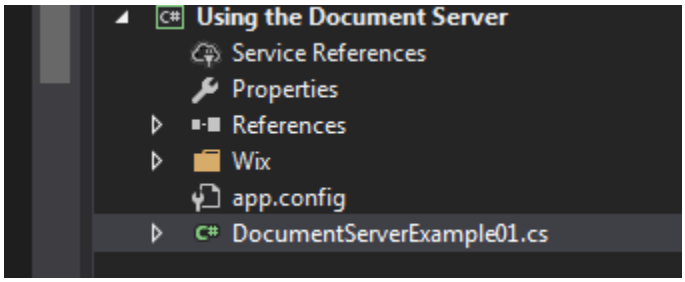
OEMResource is a PRODUCT NAME assembly.

Working with PRODUCT NAME SOAP Services (continued)

Working with PRODUCT NAME SOAP Services (continued)

Sample Code Explanation

The sample code is located in a project called ***Using the Document Server***, found in a single class file DocumentServerExample01.cs (as shown below):



The source code within this file demonstrates how to utilize the ApplicationServer DataService as well as the DocumentServer DocumentService, and performs the following actions:

- ✓ Queries the DataService
- ✓ Checks out a document from the DocumentService
- ✓ Checks in a document to the DocumentService

To execute these actions, the code performs the following detailed steps:

1) Creates proxy instances to provide connectivity to the DocumentService and DataService.
2) Sets URLs to the services for the proxies to use.
3) Authenticates a PRODUCT NAME user against the AuthenticationService and retrieves an authentication token.
4) Sets up the DocInfo object with valid data.
5) Calls the services with the DocInfo object to manipulate data within the PRODUCT NAME repository.

NOTE

If you want to build your own project that interacts with the PRODUCT NAME repository via .NET code, this project is a great starting point that should provide functionality that most systems need.

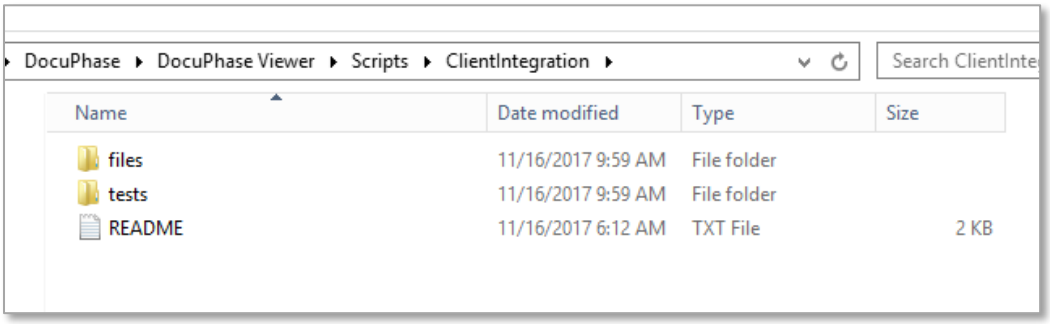
PRODUCT NAME ViewerJavaScript API

PRODUCT NAME provides an HTML5 Document Viewer capable of integration with your own web applications. Sample code and a Readme file are installed with the product on the PRODUCT NAME web server.

Access the JavaScript Client Integration Files

On your server:

1) Navigate to the location at which PRODUCT NAME is installed (this example assumes C:\Program Files\PRODUCT NAME).
2) Go to the PRODUCT NAME Viewer\Scripts\ClientIntegration directory:



- The README file contains notes about how to integrate the Viewer into your application.
- Within the tests folder, there are two files:
  - PRODUCT NAME.HTML5.Viewer.TestScript.js
  - PRODUCT NAME.HTML5.Viewer.TestPage.html
- The test page has the HTML markup and JavaScript file references needed to allow the page to interact with the PRODUCT NAME REST API and the HTML 5 Viewer:

```
<head>
    <title></title>
    <script type="text/javascript" src="../files/jquery-1.11.0.min.js"></script>
    <script type="text/javascript" src="../files/pubsub.min.js"></script>
    <script type="text/javascript" src="../files/DocuPhase.HTML5.Viewer.Client.js"></script>
    <script type="text/javascript" src="../files/DocuPhase.HTML5.Viewer.ClientBack.js"></script>
    <script type="text/javascript" src="../files/DocuPhase.HTML5.Viewer.ClientSettings.js"></script>
    <script type="text/javascript" src="DocuPhase.HTML5.Viewer.TestScript.js"></script>
    <meta charset="utf-8" />
</head>
```

The resulting page is rendered as such:

Requirements for Viewer Integration

- ✓ The hosting Web Application and the PRODUCT NAME HTML5Viewer must be installed in the same domain.
- ✓ The PRODUCT NAME DataService REST service must be installed.
- ✓ The PRODUCT NAME Security REST service must be installed.

Steps to Integrate the HTML5 Viewer

1) The JavaScript files listed below must be included in the hosting Web Application.
2) Update PRODUCT NAME.HTML5.Viewer.ClientSettings.JS to reflect your mechanism to set request information.
3) Update PRODUCT NAME.HTML5.Viewer.ClientBack.js to reflect your mechanism to respond to each Viewer function completion.
4) Update PRODUCT NAME.HTML5.Viewer.Client.js to reflect the HTML container that will be hosting the HTML5Viewer on the host application page.
5) Insert an HTML container (usually a DIV) in the host application page as specified in step 4.
6) Make sure PRODUCT NAME.HTML5.Viewer.ClientSettings.JS is called to populate the $.HTML5ViewerSettings object.
7) Call the corresponding Viewer function via the $.HTML5Viewer method passing the needed parameters (see each method description in PRODUCT NAME.HTML5.Viewer.Client.js).

File	Description
/Scripts/ClientIntegration/files/pubsub.min.js	Client side bus implementation
/Scripts/ClientIntegration/files/jquery-1.11.0.min.js	Client side JavaScript Framework
/Scripts/ClientIntegration/files/PRODUCT NAME.Viewer.Client.js	Client side ViewerJavaScript API file. For more details check the comments in the file.
/Scripts/ClientIntegration/files/PRODUCT NAME. Viewer.ClientBack.js	Provides the mechanism to hook and respond to the Viewer functions completion. Function completion available: CheckInComplete, CheckOutComplete, UndoCheckOutComplete, DeleteRevisionComplete, PromoteToTipComplete, Refresh. For more details check the comments in the file.
/Scripts/ClientIntegration/files/PRODUCT NAME. Viewer.ClientSettings.js	Provides the mechanism to set the request information. These include: Token (obtained from the Security Service), EmailPass flag (if user has e-mail password set in the PRODUCT NAME system), Data Service instance number, Viewer URL. For more details check the comments in the file.

PRODUCT NAME REST Services API

PRODUCT NAME provides RESTful API services. By utilizing the PRODUCT NAME REST services API, integrated systems are executing logic built in to PRODUCT NAME: ensuring data integrity. PRODUCT NAME REST Services API is considered to be current technology and continues to be enhanced.

All REST API endpoints include documentation that may be accessed by using the following URL pattern:

http://{server name}/{service name}/documentation

The following table shows all services and their corresponding documentation URLs:

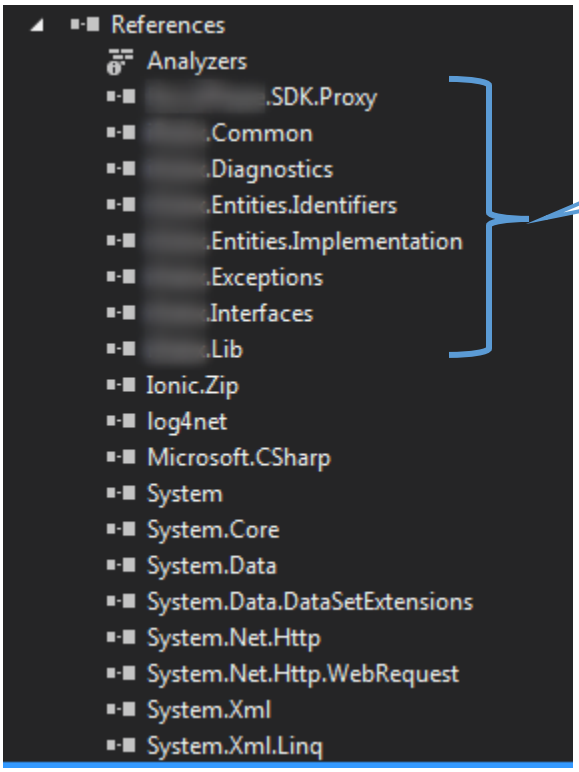
Service	Description	Documentation URL
ConfigurationService	Support for access to Configuration, Indexing and Product settings	http://{servername}/configurationservice/documentation
DataService	Support for access to information related to Documents (content, upload, versions), Forms, Annotations, OCR Data and Applications	http://{servername}/dataservice/documentation
SearchService	Support for access to search for Documents and Workflow work items	http://{servername}/SearchService/documentation
SecurityService	Support for security related activities, such as authentication, user/group maintenance and permissions	http://{servername}/SecurityService/documentation

Working with PRODUCT NAME REST API Services

The PRODUCT NAME SDK provides sample code that is located in a project called ***Using the RESTful Web Services*** that demonstrates use of the PRODUCT NAME REST API Services from C# .NET. This sample code is provided upon request.

Project References

The project references PRODUCT NAME assemblies required to compile and execute, as shown below:



Required PRODUCT NAME assemblies begin with PRODUCT NAME. naming convention.

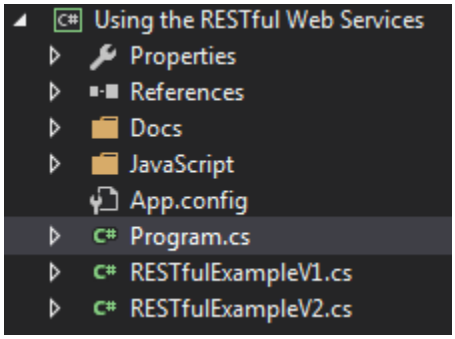
PRODUCT NAME.SDK.Proxy is the PRODUCT NAME proxy assembly that simplifies access to RESTful API services.

Working with PRODUCT NAME REST API Services (continued on next page)

Working with PRODUCT NAME REST API Services (continued)

Sample Code Explanation

The sample code located in a project called Using the RESTful Web Services project is found in three class files as shown below:



- Program.cs is the starting point for the application. This simple program instantiates classes found in RESTfulExampleV1.cs and RESTfulExampleV2.cs.
- RESTfulExampleV1 performs the following actions:
  - ✓ Logs in to PRODUCT NAME using PRODUCT NAME credentials
  - ✓ Gets applications and enumerates them
  - ✓ Creates a new document
  - ✓ Gets a document from the DataService
  - ✓ Writes the document to the local disk
  - ✓ Checks out the document
  - ✓ Modifies some document content
  - ✓ Checks in the document
  - ✓ Demonstrates use of search to find documents in PRODUCT NAME
- RESTfulExampleV2 performs the following actions:
  - ✓ Logs in to PRODUCT NAME using PRODUCT NAME credentials
  - ✓ Creates a document (non-revision controlled)
  - ✓ Updates document data

NOTE

- *These code examples make use of the PRODUCT NAME SDK Proxy that simplifies access to the REST API.*
- *Use of the SDK Proxy is not required to access the REST API.*

Questions or Need Assistance?

If you have specific questions or need assistance with the SDK or API set up, please contact Product Name at any of the following:

Email: support@Product Name.com

Phone: (727) 441-8228

Website: https://www.Product Name.com/contact-us

BEST PRACTICE

When sending an email, please use the following format:

In the Subject line: *"Request for information about Viewer SDK/API"*

In the Body of the email:
Give a brief description of the information you are looking to obtain.
Provide the best contact name, phone number, and email address.

Appendices

Appendix A – Backward Compatibility & IIS Configuration

Backward Compatibility

Although the API key/value pairs have changed, this API was designed to support API calls from prior PRODUCT NAME versions. The table below lists the changes between 5.1 and prior versions.

Previous Versions	Current Version
Current_App_ID	ApplicationID
User_ID	UserName
Logon	Login
EPass	EncrypedPassword
Pass	Password

IIS Configuration

- All prior calls were made to Results_Frame.asp. Those calls should now be set to request to HandlerPage.aspx with the appropriate QueryString values.
- Because the libraries controlling ASP and asp.net applications are different, anything referencing the older Results_Frame.asp page is not accessible from a 5.1 PRODUCT NAME instance.
- To alleviate any problems for applications that still reference Results_Frame.asp, the installer configures IIS to redirect all requests from Results_Frame.asp to HandlerPage.aspx.

  The following steps list the actions required to manually change IIS:

  1) Go to Start▶Administrative Tools▶Internet Information Services IIS Manager.
  2) Navigate to **Web Sites**.
  3) Expand the **Default Web Site** folder, then select **PRODUCT NAME** or **PRODUCT NAME**.
  4) Locate, then right-click on **Results_Frame.asp**.
  5) Select **Properties**.

  In the Results_Frame.asp properties dialog:

  6) Enable the **A Redirection to a URL** option.

  In the "Redirect to" field:

  7) Enter the following URL:

     http://SERVERNAME/PRODUCT NAME/web/handler/HandlerPage.aspx (where SERVERNAME is the name of your server).
  8) Click **Apply**.
  9) Click **OK**, then close IIS Management Console.

Appendix B – Examples of Invalid Command Line Parameters

- Invalid username, password or Web server.
- Application ID parameter missing, not found in PRODUCT NAME database or application referenced by the application ID is not "implemented" in PRODUCT NAME.
- Index field name/value parameter (-i) is not passed using the following format: –iIDX_name=fieldvalue.
- Index field name does not exist within the application ID.
- Index field value is invalid. The program tests for numeric, decimal and date/time values if applicable for the past index field name.
- -iIDX_TestDate=”13/04/2008” is an invalid date.
- -iIDX_TestDate=”14/29/2008” is a valid date